



EVT: Accelerating Deep Learning Training with Epilogue Visitor Tree

Zhaodong Chen
UC Santa Barbara
United States
chenzd15thu@ucsb.edu

Jack Kosaian
NVIDIA
United States
jkosaian@nvidia.com

Andrew Kerr
NVIDIA
United States
akerr@nvidia.com

Haicheng Wu
NVIDIA
United States
haichengw@nvidia.com

Richard Cai
NVIDIA
United States
ricai@nvidia.com

Yufei Ding
UC San Diego
United States
yufeid@ucsd.edu

Yuan Xie
HKUST
China
yuanxie@ust.hk

Abstract

As deep learning models become increasingly complex, the deep learning compilers are critical for enhancing the system efficiency and unlocking hidden optimization opportunities. Although excellent speedups have been achieved in inference workloads, existing compilers face significant limitations in training. Firstly, the training computation graph involves intricate operations challenging to fuse, such as normalization, loss functions, and reductions, which limit optimization opportunities like kernel fusion. Secondly, the training graph's additional edges connecting forward and backward operators pose challenges in finding optimal and feasible partitions for kernel fusion. More importantly, existing compilers cannot either generate kernels with state-of-the-art performance on modern GPUs or accommodate diverse fusion patterns.

In this paper, we introduce Epilogue Visitor Tree (EVT), a novel compiler that overcomes these limitations. EVT employs novel graph-level compilation passes to unlock hidden fusion and optimization opportunities. It also incorporates a

novel integer linear programming-based partitioner that efficiently solves the optimal and feasible partitions in complex joint forward-backward graphs. Moreover, we present the Epilogue Visitor Abstraction and introduce the EVT operator compiler that automatically generates flexible epilogues that can be integrated with high-performance main loop implementations from CUTLASS and other SOTA libraries. EVT is evaluated on diverse training workloads across domains and achieves 1.26~3.1 $\times$ speedup.

ACM Reference Format:

Zhaodong Chen, Andrew Kerr, Richard Cai, Jack Kosaian, Haicheng Wu, Yufei Ding, and Yuan Xie. 2024. EVT: Accelerating Deep Learning Training with Epilogue Visitor Tree. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (ASPLOS '24)*, April 27-May 1, 2024, La Jolla, CA, USA. ACM, New York, NY, USA, 16 pages. <https://doi.org/10.1145/3620666.3651369>

1 Introduction

The rapid evolution of deep learning has led to a surge in the complexity and quantity of deep learning models. These models, often implemented through deep learning frameworks like PyTorch [22] that target flexible programming, face suboptimal performance as they cannot efficiently utilize the resources provided by hardware like GPUs. Manual optimization of these models demands a high degree of engineering effort and expertise, and even experts may overlook hidden optimization opportunities within intricate models.

Deep learning (DL) compilers play a critical role in addressing this challenge. A typical DL compiler optimizes the user-defined model through three major steps [16]: graph-level optimization, partition, and operator-level optimization. In detail, the input model is first traced into a graph representation, with nodes representing operators and edges denoting dataflow. Graph-level optimization simplifies the graph by

Work partially done during the internship at NVIDIA

The source code to reproduce the results is publically available at https://github.com/apuaaChen/EVT_AE. The CUDA templates of EVT are actively maintained under the official CUTLASS repository <https://github.com/NVIDIA/cutlass>.



This work is licensed under a Creative Commons Attribution International 4.0 License.

ASPLOS '24, April 27-May 1, 2024, La Jolla, CA, USA

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0386-7/24/04.

<https://doi.org/10.1145/3620666.3651369>

folding constants or eliminating common sub-expressions. Subsequently, the partitioner divides the graph into individual partitions, and each partition is further optimized by operator-level optimization, such as replacement by a fused kernel generated by the operator compiler [28].

While considerable progress [4, 22, 26] has been made in optimizing models for inference efficiency, three key limitations remain in optimizing the training workload.

Limitation 1. Existing operator compilers are inadequate to generate fused kernels that deliver state-of-the-art performance while accommodating diverse fusion patterns. Loop-based compilers, like TVM [4], represent operators as nested loops. They face challenges in abstracting dedicated optimizations and leveraging the accelerator features and datapaths in modern GPUs for core operators like matrix multiplication (GEMM) and convolutions. On the other hand, template metaprogramming (TMP)-based compilers [29, 30] that leverage expert-developed template libraries lack the flexibility to support diverse fusion patterns in the training graphs.

Limitation 2:. While existing studies focus on the forward and backward passes, few optimizations for loss functions have been proposed. However, in scenarios such as extreme classification [17], the loss functions and their corresponding backward pass can become performance bottlenecks.

Limitation 3. Existing partitioning algorithms cannot find feasible and optimal partitions in the training graph. Unlike inference, which contains a stream of operators and can be partitioned with simple heuristics, the training graph includes additional edges passing saved activations to backward operators for gradient computation. Heuristically partitioning them can easily lead to an infeasible result that contains cycles. While arbitrarily separating the forward and backward graph obscures opportunities for fusing operators from both forward and backward parts. Moreover, existing partitioning algorithms are tailored to specific operator compilers, lacking extensibility to integrate additional optimization targets, heuristics, and constraints.

1.1 Our Approach

We present Epilogue Visitor Tree (EVT), a novel compiler designed to address all the aforementioned limitations.

The core of this paper is the Epilogue Visitor Tree abstraction, designed to tackle limitation 1. The key idea is composing fused kernels via mainloop-epilogue fusion, akin to TMP-based compilers, while addressing the scalability of the epilogue in TMP-based compiler through a visitor design allowing epilogue operators to be developed, optimized, and verified individually, and then assembled under the tree structure by compiler. With this abstraction, the expert-designed mainloop ensures state-of-the-art performance as all the dedicated optimizations written in CUDA are automatically incorporated, the compiler assembled epilogues offer strong flexibility to handle diverse fusion patterns in training.

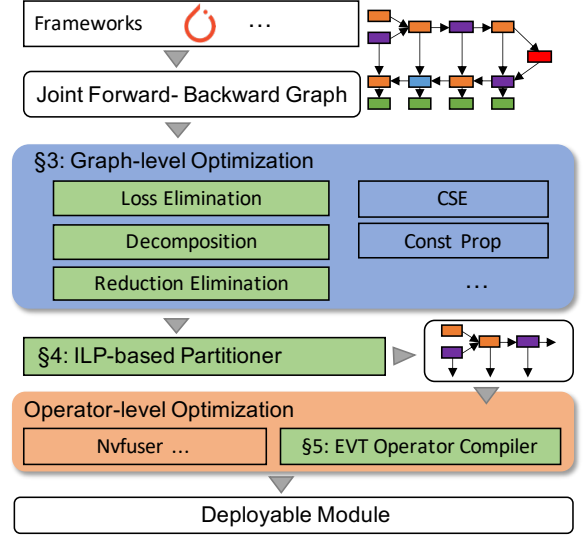


Figure 1. Overview of the EVT System Design.

Although the Epilogue Visitor Tree abstraction offers strong flexibility, it still imposes constraints on fusible patterns. For instance, only supported epilogue operators can be fused. The fusion between reductions and their consumers in the epilogue is also challenging and sometimes not beneficial. These constraints are mitigated by the co-designed graph-level optimizations and partitioning algorithm.

Figure 1 provides an overview of EVT. It directly takes models from frameworks like PyTorch [22] and converts them into a joint forward-backward graph, where training samples and model parameters serve as inputs, and gradients are computed as outputs.

Graph-level Optimization. To address Limitation 2, EVT optimizes the loss function and its backward pass through "Loss Elimination". This approach unlocks the opportunity to fuse the loss function and its backward pass by recognizing the loss value doesn't participate in the backward computation.

To mitigate the constraints of EVT abstraction, "Decomposition" is employed to break down complex, non-fusible operators like *batch_norm* into operators that are fusible under the EVT abstraction. Additionally, "Reduction Elimination" helps alleviate the reduction fusion constraint by identifying instances where the reduction result can be expressed equivalently as a series fusible operators.

Partitioning. Limitation 3 is addressed by our ILP-based partitioner. It models the partitioning as an ILP problem to find the feasible global optimal solution. This formulation also allows for the integration of additional optimization targets, heuristics, and constraints. This flexibility enables the encoding of EVT abstraction constraints unsolved by the graph-level optimization into the ILP solver's constraints, ensuring all the solved partitions can be fused.

As the acyclic partitioning problem is NP-Hard [21], we propose several simplification techniques to reduce the complexity, and large-scale neural networks with millions of decision variables can be solved in minutes.

Operator Compiler. Each partition involving heavy operators like GEMM will be dispatched to our EVT Operator Compiler. It is flexible enough to support diverse fusion patterns in training while generating fused kernels that deliver state-of-the-art performance.

Finally, EVT returns a deployable model to the user, which is compatible with other optimization techniques such as CUDA graph to perform further optimizations orthogonal to ours. Our EVT can be used as a backend for *torch.compile* that optimizes a model with a single line of code.

2 Related Work

2.1 Graph-level Optimization

Existing DL compilers have adopted various graph-level optimizations to improve the efficiency of Deep Learning Systems. TVM [4] optimizes its graph-level IR, *relay*, with passes like constant-folding, decomposition, and data layout transformation.

However, few optimizations have been proposed for the loss function and its backward pass, which could be a performance bottleneck in many cases. Additionally, existing graph-level optimizations, such as decomposition, are typically designed independently with the underlying operator-level optimization. This approach misses the opportunity to address certain limitations of operator-level optimization at the graph level.

2.2 Partitioning Algorithms

Most existing studies employ simple heuristics when partitioning computation graphs. TVM[4], for instance, classifies the operators into four types: injective (element-wise), reduction, complex out-fusible (can fuse element-wise map to output), and opaque (cannot be fused). They define heuristic rules such as reduction can be fused with input injective operators and complex out-fusible nodes like GEMM can fuse element-wise operators to its output. However, as we will illustrate in Section 4, these heuristics would lead to infeasible partitions with cycles or suboptimal solutions. Additionally, existing partitioning algorithms lack the flexibility to incorporate additional optimization objectives, heuristics, and constraints.

There are also studies on the acyclic partitioning problem beyond the Deep Learning System community [9]. Our ILP-based partitioner takes inspiration from Nosack and Pesch [19] which formulates the acyclic partitioning problem as ILP. The ILP-based formulation provides great extensibility to encode new constraints and heuristics through ILP constraints. However, the high complexity of the approach developed by Nosack and Pesch [19] makes it impractical to

be directly used when solving large-scale neural networks with thousands of nodes, and their lower bound obtained from Kernighan’s solution [13] is not valid when additional constraints are involved. In Section 4, we develop a novel approach dividing the computation graph into smaller ones so that they can be solved in a reasonable amount of time.

2.3 Operator-level Optimization

Kernel fusion is the key optimization applied in operator-level optimization. During kernel fusion, multiple operators under the producer-consumer relationship are fused into a single one. The intermediate result can be directly cached in the register or shared memory to reduce memory access. The performance of the fused kernel is determined by the quality of the implementation of each operator, while the flexibility is determined by the ability of the operator compiler to align the loop structure of different operators.

Template meta-programming (TMP)-based compilers, such as AITemplate [30] and Bolt [29], use expert-developed template libraries [5, 10, 25] like CUTLASS to achieve optimal performance. However, existing TMP-based compilers lack the flexibility to cover the diverse fusion patterns encountered in the training workloads. They only support a limited set of patterns defined by the template library, posing challenges for scalability.

Loop-based compilers, such as TVM [4], Tiramisu [1], NvFuser, and Tensor Comprehensions [27], represent operations as loops and apply schedules, such as loop fission, fusion, parallel, and vectorization, to map them to GPUs. The loop-based compilers have demonstrated remarkable success in inference tasks and outperform state-of-the-art libraries crafted by experts. The loop-based abstraction can easily divide the workload under small problem size to adequate threadblocks, addressing the low utilization of streaming multiprocessors (SMs) of GPU.

In contrast, the training workloads have sufficiently large batch sizes to saturate the SMs, and the kernels’ performance is bounded by computation and memory utilization. Improving these utilization requires dedicated and sometimes case-by-case optimizations with CUDA, which are challenging to express as transformations on IRs.

Therefore, existing approaches like inductor of PyTorch [22] still rely on kernels from cuDNN or templates from CUTLASS [25] for heavy operators like GEMM, despite their ability to generate kernels.

3 Graph-level Optimizations

In this section, we first discuss the major optimizations exploited and then provide a running example to illustrate how these optimizations simplify the computation graph, particularly the loss function and its backward pass, and expose new fusion opportunities.

3.1 Optimizations

Loss Elimination. Despite it sounding counter-intuitive, we found that the loss value can be eliminated as a dead node from the joint forward-backward graph for two reasons: the backward pass can be performed without computing the loss value, and the loss value is not required for every iteration.

For the first reason, the loss value does not participate in the computation of the backward pass. For example, with batch size n , in the widely used softmax cross-entropy loss function, the gradient of logits X under target Y can be simplified to

$$\frac{\partial L}{\partial X} = -\frac{1}{n} \text{OneHot}(Y) + \frac{1}{n} \text{Softmax}(X). \quad (1)$$

With this equation, the gradient can be directly computed with higher numerical stability without the loss value.

For the second reason, the loss value is required only when the users need to observe it to assess the training process. However, the training process can take millions of iterations, and it is not necessary to check the loss of each iteration. Therefore, we can compute the loss only when it is required and use the accelerated version for the remaining iterations.

With this domain knowledge, we propose loss elimination that eliminates the loss value as a dead node. However, existing DNN frameworks like PyTorch still use the loss value to trigger the backpropagation, so we achieve this by simply replacing the loss value with a constant scalar tensor. Under the hood, loss elimination bypasses the reduction that calculates the scalar loss. It transforms both forward and backward passes of loss function into element-wise operations, thereby creating new fusion opportunities and improving numerical stability.

Decomposition. The deep learning frameworks contain many non-fusible operations like *logSoftmax*, *addmm*, designed to simplify programming and leverage existing fused kernels. However, these operations also become the major obstacle for kernel fusion, as each of them may require special epilogue operators to handle. To tackle this issue, we register decomposition rules for these operations, breaking them down into basic fusible operations like element-wise operations and reductions through pattern matching and rewriting. Table 1 provides examples of these decomposition rules.

One different of these decomposition rules with existing studies is that they are designed by taking the fusion constraints of the EVT operator compiler (Section 5.2) into consideration. On one hand, the decomposition results are composed of operators supported by the EVT operator compiler when possible. On the other hand, the dependency of reduction results are tried to avoided. For instance, the decomposition of Batch Normalization (BN) [11], inspired by Jung et al. (2019)[12], leverages the equation $\sigma = \sqrt{\sum X^2/n - (\sum X/n)^2}$ to decouple the two reductions, enabling more fusion opportunities.

Table 1. Decomposition Examples in EVT

Pattern	Decomposition
$\text{logSoftmax}(X)$	$\text{log}(\text{Softmax}(X))$
$\text{nllLossBp}(X, Y)$	$-\frac{1}{n} \text{OneHot}(Y)$
$\text{logSoftmaxBp}(dY, X)$	$dY - \text{logSoftmax}(X) \sum dY$
$\text{dropout}(X, p)$	$\frac{X \odot (\text{rand_like}(X) > p)}{1-p}$
$\text{dropoutBp}(dY, M, p)$	$\frac{dY \odot M}{1-p}$
$\text{reluBP}(dY, X)$	$dY \odot (X \geq 0)$
$\text{addmm}(X, W, b)$	$\text{mm}(X, W) + b$
$\text{BN}(X, \alpha, \beta)$ [12]	$\mu = \sum X/n, m2 = \sum X^2/n$ $\sigma = \sqrt{m2 - \mu^2}$ $\frac{\gamma}{\sigma}(X - \mu) + \beta$
$\text{BNBp}(dY, X, \mu, \sigma, \gamma)$ [12]	$d\beta = \sum dY, \hat{X} = \frac{X - \mu}{\sigma}$ $d\gamma = \sum (dY \odot \hat{X})$ $dX = \frac{\gamma}{\sigma}(dY - \frac{d\beta}{n}) - \frac{d\gamma \gamma \hat{X}}{n\sigma}$

Besides these registered decomposition rules, we also provide a simple interface for users to register custom rules by declaring the pattern and replacement as normal Python functions. We provide an example as follows:

```
def pattern(bias, x, weight):
    return torch.ops.aten.addmm(bias, x, weight)

def replacement(bias, x, weight):
    mm = torch.ops.aten.mm(x, weight)
    return torch.ops.aten.add(mm, bias)
```

Reduction Elimination. Similar to existing operator compilers, our EVT operator compiler faces challenges in fusing reductions with their consumers, thereby limiting fusion opportunities. This limitation arises from the need for special parallelization, which requires all elements to be reduced to be partitioned to the same threadblock. This parallelization is typically incompatible with the mainloop tiling of operators like GEMM.

In certain scenarios, a reduction operation can be transformed into a sequence of fusible element-wise operators when it involves operators with a constant reduction value. For instance, with integers $y_{0 \leq i \leq m} \in [0, n)$ and a scalar α

$$\sum_{j=0}^{n-1} \alpha \text{OneHot} \left(\begin{bmatrix} y_0 \\ \vdots \\ y_m \end{bmatrix} \right)_{:,j} = \alpha \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}. \quad (2)$$

We introduce reduction elimination that automatically identifies these opportunities and rewrites the graph to unlock additional fusion opportunities. A major challenge is that operations with constant reduction values may not always be the direct input to the reduction. There could be a sequence of associable arithmetic operations (+, -, ×, ÷) between them. Following existing approaches in algebraic reassociation of expression, we address this challenge with 3 steps:

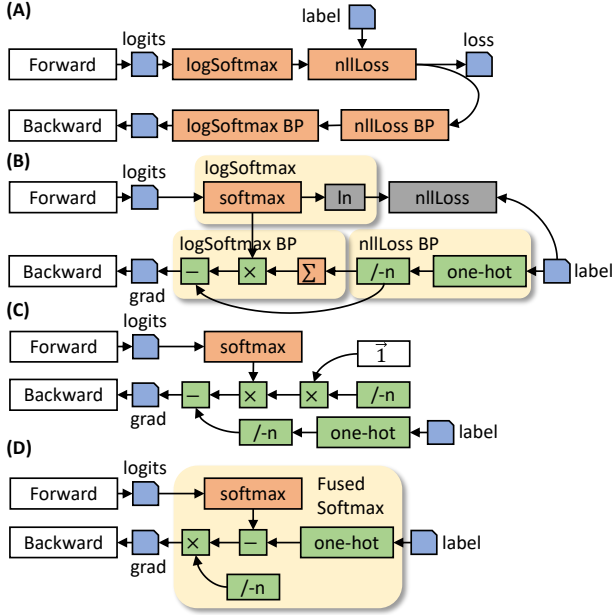


Figure 2. Optimization of Softmax Cross-Entropy Loss.

- Extract the subgraph generating the reduction, containing only operators associable with the reduction. When encountering a non-associable node, replace it with a placeholder and annotate it if it has constant reduction value.
- Perform expression reassociation following Briggs et al. (1994) [2], assigning rank -1 to reduction nodes such that they have the lowest rank. After reassociation, the reductions always take a placeholder as input. If the placeholder has a constant reduction value, fold it to a static tensor.
- Conduct constant propagation, if the subgraph contains no reduction nodes, substitute it back to the original graph.

Other Graph Simplifications. In addition to the specialized optimizations discussed earlier, we also included a set of standard graph-level optimizations to further simplify the computation graph. These include conventional techniques such as constant propagation with expression reassociation, common factor extraction, common sub-expression elimination, and dead code elimination.

3.2 Example

To illustrate how aforementioned optimizations simplify the computation graph and uncover new fusion opportunities, we present a running example here that shows how the *softmax_cross_entropy* and its backward operations can be simplified to a single fused softmax that not only enhances numerical stability but also improves the overall performance.

The *softmax_cross_entropy* is a standard loss function widely adopted by the deep learning community. It can be a performance bottleneck in many cases. For instance, our profiling shows that when training Graph Convolutional Neural

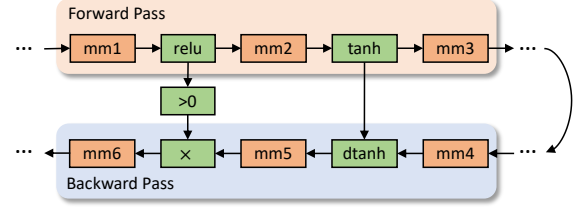


Figure 3. Illustrative Example for Partition.

Networks (GCNs) [14] on ogbn-mag dataset, the loss and its backward pass contribute over 13% of the total iteration time. This inefficiency comes from that large reduction along the batch dimension, which is unfriendly to mixed precision training due to numerical stability concerns.

Figure 2 illustrates the traced computation graph. It contains four non-fusible nodes. After our decomposition with patterns in Table 1, these non-fusible nodes are broken down into fundamental fusible nodes. Then, our loss elimination eliminates the *nllLoss* as a dead node. While the Σ node, decomposed from *logSoftmax BP*, initially hinders fusion with the decomposed ops from *nllLossBP*, our reduction elimination pass folds it to basic element-wise operations. Finally, the other graph simplification passes clean up the computation graph, eventually enabling it to be fused by our operator compiler. As the optimized computation graph has no reductions, and all the intermediate results are stored in registers with full precision, it has much better numerical stability compared with the original version.

4 Partitioner

With the optimized graph in place, the next step is dividing it into disjoint partitions for fusion. We first use a simple example in Figure 3 to demonstrate heuristics used by existing studies may fail to find feasible and optimal partitions.

Example. Figure 3 shows the joint forward-backward graph of two layers. The element-wise operators are colored green, *mm*s stand for matrix multiplications that can fuse element-wise operators to its output. For feasibility, $\{mm2, tanh, dtanh\}$ is valid under TVM[4]’s rule, yet it is infeasible due to the cycle. In terms of optimum, while $\{mm1, relu\}$ and $\{mm5, > 0, \times\}$ forms a valid partition, it is suboptimal as partition $\{mm1, relu, > 0\}$ and $\{mm5, \times\}$ saves more memory access with the output of “> 0” stored under bitmask.

Our Method. We present a novel algorithm that formulates partitioning as an integer linear programming problem that offers three key benefits:

- **Feasibility:** The requirements including acyclic are encoded as ILP constraints.
- **Optimum:** The objective function maximizes saved memory access through fusion in each partition.
- **Extensibility:** Additional requirements and heuristics can be easily encoded as constraints.

With the "Extensibility", the partitioner can easily incorporate constraints and heuristics from the EVT abstraction and custom objectives.

In this section, we first discuss the ILP formulation following Nossack and Pesch [19], then present our novel techniques to reduce its solving time.

4.1 Integer Linear Programming Formulation

Let $D = (V, A)$ be the topological sorted computation graph with node set $V = \{v_1, \dots, v_n\}$ and edge set $A \subseteq \{(v_i, v_j) | v_i, v_j \in V\}$. The objective is to find disjoint partitions $\{V_1, \dots, V_k\}$ of D under certain constraints and heuristics, such that each partition can be fused. The benefit is the sum of memory access caused by the internal edges within each partition. Therefore, we assign each edge an edge weight $c_{ij} \in \mathbb{N}^+$ describing the size of the tensor passed long it. Following Nossack et al. [19], four types of decision variables are defined:

- $x_{is} \in \{0, 1\}$: $x_{is} = 1$ when node $v_i \in V$ in cluster V_s .
- $z_{ijs} \in \{0, 1\}$: $z_{ijs} = 1$ indicates both nodes $v_i, v_j \neq i \in V$ belong to cluster V_s .
- $y_{st} \in \{0, 1\}$: $y_{st} = 1$ identifies there is an edge between the nodes of cluster V_s and $V_t \neq s$.
- $\pi_s \in \mathbb{Z}$: auxiliary variable used to formulate the Miller-Tucker-Zemlin(MTZ) subtour elimination constraints that ensure acyclic partitioning [18].

The ILP problem can be formulated as follows. The object function (3a) maximizes the sum of edge weights internal to each partition. Constraint (3b) ensures that each node is exclusively partitioned to a partition. Constraints (3c) and (3d) encode additional constraints and heuristics. Constraints (3e) and (3f) connect the decision variables x , y , and z . Constraint (3g) formulates the acyclic constraint. Constraint (3h) reduces the symmetry of the solutions. At last, Constraints (3i) and (3j) are the bounds of each decision variable.

$$\max \sum_{1 \leq s \leq n} \sum_{1 \leq i \leq j \leq n} (c_{ij} + c_{ji}) \cdot z_{ijs} \quad (3a)$$

$$\text{s.t.} \sum_{s=1}^n x_{is} = 1, \forall 1 \leq i \leq n \quad (3b)$$

$$x_{is} + x_{js} \leq 1, \forall 1 \leq s \leq n, v_i \text{ cannot fuse with } v_j \quad (3c)$$

$$\sum_{s=1}^n z_{ijs} = 1, v_i \text{ must fuse with } v_j \quad (3d)$$

$$z_{ijs} \leq x_{is}, z_{ijs} \leq x_{js}, \forall 1 \leq i < j \leq n, 1 \leq s \leq n \quad (3e)$$

$$x_{is} + x_{jt} - 1 \leq y_{st}, \forall (v_i, v_j) \in A, 1 \leq s \neq t \leq n \quad (3f)$$

$$\pi_s - \pi_t + n \cdot y_{st} \leq n - 1 \forall 1 \leq s \neq t \leq n \quad (3g)$$

$$\sum_{i=1}^n x_{is} \leq \sum_{i=1}^n x_{i,s-1} \forall 2 \leq s \leq n \quad (3h)$$

$$x_{is}, z_{ijs}, y_{st} \in \{0, 1\} \forall 1 \leq i < j \leq n, 1 \leq s \neq t \leq n \quad (3i)$$

$$\pi_s \in \mathbb{Z}, \forall 1 \leq s \leq n \quad (3j)$$

Algorithm 1: Dividing Node Set

Input: Computation Directed Acyclic Graph $D = (V, A)$.
 A list of edges $A_{ne} \subset A$ such that $\forall (v_i, v_j) \in A_{ne}$, v_j cannot be fused into v_i .
 A list of nodes pairs A_{nv} , $(v_i, v_j) \in A_{nv}$ indicates v_i and v_j cannot be fused

Output: Disjoint components $\{C_1, \dots, C_m\}$.

- 1 Create the directed graph $\bar{D} = (V, A/A_{ne})$
- 2 Construct a dict R that maps each node $v_i \in V$ to all the nodes it can reach through directed edges in D .
- 3 **for** $v_i \in V$ **do**
- 4 Unfusible Set $S_i = \{\}$
- 5 **for** $v_j \in R[v_i]$ & $(v_i, v_j) \in A_{nv}$ **do**
- 6 $S_i = S_i \cup \{v_j\} \cup R[v_j]$
- 7 **for** $v_k \in S_i$ **do**
- 8 remove edge (v_i, v_k) from $\bar{D}$ if exists
- 9 $\{C_1, \dots, C_m\} = \text{weakly_connected_components}(\bar{D})$

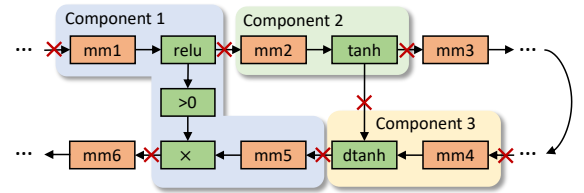


Figure 4. Motivating examples for Dividing Node Set.

4.2 Reduce the Complexity

While formulation (3) offers feasibility, optimum, and high extendability, solving it directly is impractical. The four types of variables introduce $O(n^3)$ decision variables, and the branch and bound method further takes exponential time in the worst case to find the optimal solution. To address this challenge, inspired by the fact that neural networks are constructed by stacking similar layers, we divide D to disjoint components and solve Equation (3) separately in each component. Additionally, we cache the solutions to avoid solving the same ILP problem twice for components with similar structures. The major challenge is ensuring that the combined solution from each component remains both feasible and optimal. We present a two-step solution as follows:

- Divide node set V to disjoint components $\{C_1, \dots, C_m\}$ without hurting optimum.
- Reconstruct the edges in each component that ensures the feasibility of the solution and solve Equation (3).

Dividing Node Set. We divide the node set with Algorithm 1. Our insight is that in the optimal solution, nodes belong to the same partition only if they are weakly connected by fusible edges. Thus, we cut all non-fusible edges in D and divide it into weakly connected components. Figure 4 shows that this approach divides D into multiple small

Algorithm 2: Reconstructing Edges and Solving ILP of Each Component

Input: Computation Graph $D = (V, A)$. $\tilde{D} = (V, \tilde{A})$ and components $\{C_1, \dots, C_m\}$ from Algorithm 1.

Output: Partitions $\{V_1, \dots, V_k\}$, Bool indicator $is_optimal$

```

1 Create graph  $\tilde{D} = (V, A + \tilde{A}^T)$ .
2 Partitions =  $\{\}$ ,  $is\_optimal = true$ 
3 for  $C_i \in \{C_1, \dots, C_m\}$  do
4   Get the directed graph  $\tilde{D} \setminus C_i$  that excludes nodes in  $C_i$ .
5   Get the one-hop ancestors  $N_i^a$  and descendants  $N_i^d$ 
    $N_i^a = \{v_j | v_j \notin C_i, \exists v_k \in C_i, (v_j, v_k) \in A\}$ 
    $N_i^d = \{v_j | v_j \notin C_i, \exists v_k \in C_i, (v_k, v_j) \in A\}$ 
6   Extract the subgraph  $D_i$  containing  $V_i' = C_i \cup N_i^a \cup N_i^d$ :
    $D_i = \{V_i', A_i' = \{(v_i, v_j) | (v_i, v_j) \in A, v_i, v_j \in V_i'\}\}$ 
7   for  $v_d \in N_i^d$  do
8     for  $v_s \in N_i^a$  do
9       if  $has\_path(\tilde{D} \setminus C_i, v_d, v_s)$  then
10        if not  $\exists path v_s \rightarrow v_d$  in  $D$  including nodes
11        from at least two components then
12          add_edge( $D_i, (v_d, v_s)$ )
13          if  $has\_path(D \setminus C_i, v_d, v_s)$  then
14             $is\_optimal = false$ 
15   Solve Equation (3) with additional constraint that each node in
    $N_i^a$  and  $N_i^d$  form a partition with size 1.
16   for  $n \in [1, |C_i|]$  do
17      $V_n = \{v_n | x_{nk} == 1\}$ 
18     if  $V_n \neq \emptyset$  then
19       Partitions.append( $V_n$ )

```

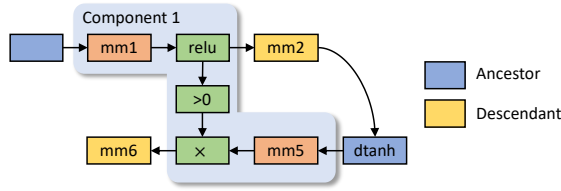


Figure 5. Example of Reconstructing Edges for Feasibility.

components with few nodes while leaving the partition of component 1 to the ILP solver.

Two types of edges are identified as non-fusible in Algorithm 1. The first (line 1) includes edges directly connecting two non-fusible nodes, such as the incoming edges to *mm*s. The second type (line 2-8) involves edges that, if fused, would create cycles. We leverage a necessary condition from Theorem 3.3 in Nossack et al. [19]: if a directed path from v_i to v_j exists in D and v_i, v_j belong to different partitions, then all nodes that can be reached from node v_j cannot be in the same partition with v_i . In Figure 4, the edge between "*tan**h*" and "*dtan**h*" is cut based on this rule, as "*tan**h*" cannot be fused with "*mm**3*" while "*mm**3*" can reach "*tan**h*".

Reconstructing Edges and Solving ILP. With Algorithm 2, for each component, we reconstruct the edges to

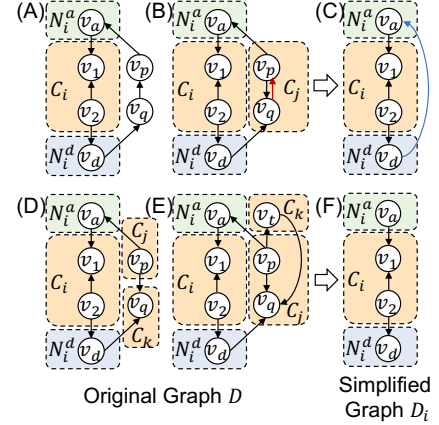


Figure 6. Adding edges from descendants to ancestors.

ensure the feasibility before solving the ILP. Theorem 4.1 below guarantees its solutions are always feasible, and they are optimal under an easily verifiable condition, which always holds during our evaluation of various DL models.

Theorem 4.1. *Algorithm 2 returns the global optimal and feasible solution when $is_optimal$ is true. Otherwise, it returns a feasible solution. (Proof: See Supplemental Material).*

Figure 5 uses component 1 from Figure 4 as an illustrative example. A node is a descendant if it has an incoming edge from the component (e.g. "*mm**2*"), and it is an ancestor if it has an outgoing edge to the component (e.g. "*dtan**h*"). We construct the edges in 3 steps:

- Identify all the ancestors and descendants of the current component (line 5,6).
- Modify D by replacing any edges that can be fused with bidirectional edges (line 4, 10).
- Add edges from any descendants to any ancestors if a path exists in the modified graph between them (line 11), e.g. the edge $mm2 \rightarrow dtanh$ in Figure 5.

The intuition is that the solutions from Figure 5 are feasible and optimal if the acyclic constraint is neither loosened nor tightened. In other words, a cycle exists in Figure 4 involving nodes in component 1 is a sufficient and necessary condition for a cycle exists in Figure 5.

Ancestors and descendants serve as auxiliary nodes. If a fusion in component 1 creates a cycle involving nodes outside the component, the cycle must contain pairs of descendants and ancestors. We then use these as boundaries to break the cycle into paths within and outside the component. The condition becomes: a path exists in Figure 4 between any pair of descendants and ancestors is a sufficient and necessary condition for a path to exist in Figure 5 with the same source and target.

Internal paths from ancestors to descendants are ensured sufficient and necessary by preserving all internal edges.

However, external paths from descendants to ancestors can be influenced by fusion in other components. In detail, while fusion does not remove paths, it creates new ones equivalent to changing fused edges to bidirectional. For example, fusing v_p and v_q in Figure 6 (B) creates a path from $v_d \rightarrow (v_p, v_q) \rightarrow v_a$ equivalent to adding the red arrow. To ensure feasibility, we add edges from any descendant to any ancestor in Figure 5 when a path could exist between them. Figure 6 illustrates four different scenarios. Figure 6 (A,B) shows scenarios when the edge should be added. In (A), there is a natural path $v_d \rightarrow v_a$. In (B), v_p and v_q belong to the same component and could be fused, which creates the path $v_d \rightarrow (v_p, v_q) \rightarrow v_a$. Oppositely, Figure 6 (D,E) shows cases when the edge is not needed. In (D), v_p and v_q belong to different components, so they cannot be fused. In (E), although v_p and v_q belong to the same component, fusing them is infeasible as it creates a cycle $(v_p, v_q) \leftrightarrow v_k$. While this method ensures the condition is necessary, it can also be sufficient (optimal) if whenever a path exists in the modified graph, there is always a path that exists in the original graph. Line 12-13 checks this condition and returns indicator *is_optimal*.

Caching the Solutions. We store the ILP solutions in a database so that they can be retrieved for components with isomorphic structures. Particularly, keys are created with the Weisfeiler Lehman graph hashing [24] so that isomorphic components will have identical hash.

5 Operator-level Optimization

In this section, we first explain the key abstraction under our operator compiler, "Epilogue Visitor". This abstraction allows fusing operators under the producer-consumer relationship by directly leveraging expert-designed implementations for the producer part and ensuring optimal performance while assembling the consumer part at compile time to accommodate to different fusion patterns. Under the abstraction, we present our EVT operator compiler, which automatically generates fused kernels with state-of-the-art performance while supporting diverse patterns.

5.1 Epilogue Visitor Abstraction

The Epilogue Visitor abstraction originates from the Mainloop-Epilogue abstraction widely adopted by existing template libraries and TMP-based compilers [25].

Mainloop-Epilogue Abstraction and Its Limitation. As shown in Figure 7(A), the Mainloop-Epilogue abstraction fuses operators under the producer-consumer relationship. It requires the consumer to have the same set of spatial loops as the producer so that they can be fused by injecting the statement of the consumer into the end of the producer's spatial loops. Subsequently, As shown in Figure 7(B), the loops are restructured to achieve optimal performance through techniques such as tiling, parallelization, and software pipelining. The Mainloop-Epilogue abstraction takes the producer as

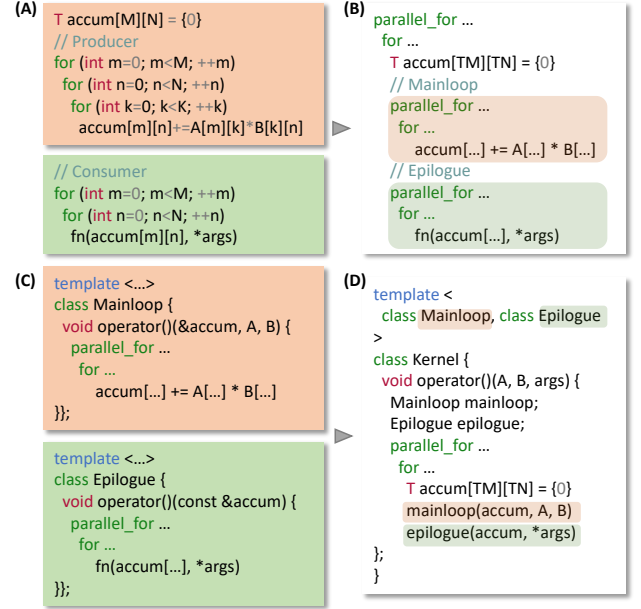


Figure 7. Mainloop-Epilogue Abstraction

the mainloop and the consumer as the epilogue. As shown in Figure 7(C), different mainloops and epilogs are developed separately as different C++ classes, and a universal kernel template in Figure 7 (D) takes the mainloop and epilogue as template arguments. This modular approach allows assembling different mainloop-epilogue pairs at compilation time and supports $|mainloop| \times |epilogue|$ patterns.

However, the "Epilogue" of the Mainloop-Epilogue abstraction is not scalable as the number of consumer operators grows rapidly. It is impractical to implement all potential epilogs manually, optimizing and verifying intricate epilogs also poses significant challenges.

Epilogue Visitor Abstraction. Our EVT abstraction preserves the mainloops and further modularizes the epilogue to address its scalability issue. As shown in Figure 8 (A,B), We model the consumer as a computation graph instead of a single operator, our key insight is that the rapid growth of potential consumer operators occurs on the **edge set** instead of the **node set**, i.e. different consumers have different combinations of the same set of basic operators. This insight motivates us to manage a library of high-performance implementation of basic operators decoupled from each other, and develop a compiler to assemble them at compile time for different consumers. The detailed explanation is as follows.

Supported Patterns. EVT supports any consumers that can be written as a perfectly nested loop, where all the statements are in the spatial loops of the producer (Figure 8 (A)). This structural constraint has proven to be key in simplifying transformations and has been widely adopted by many state-of-the-art compilers like the *linalg* dialect of MLIR [15]. This requirement is loose enough to capture most scenarios in

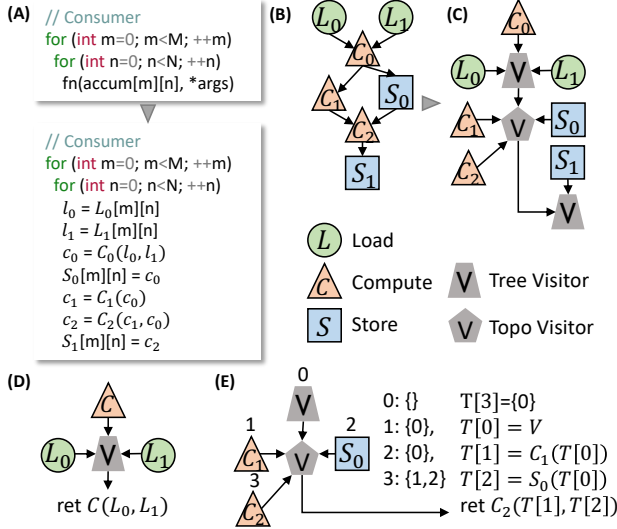


Figure 8. Epilogue Visitor Tree Abstraction

neural networks, including element-wise load/store, broadcast, reduction (atomic), and element-wise computations.

Decouple Epilogue Operators with Visitor. Figure 8 shows the perfectly nested loop representation of the epilogue, where the original function can be decomposed into a series of load, store, and compute operators. We visualize it as a graph in Figure 8(B). Notably, the store is modeled as a transparent node that forwards its input. To decouple the implementation of each node, we introduce the "visitor" and rewrite the graph to Figure 8(C), where all nodes become leaves. Two types of visitors are introduced. The tree visitor in Figure 8(D) handles non-leaf nodes with output degree 1. The topological visitor in Figure 8(E) takes other general cases. It sorts the inputs in topological order and creates an edge list describing the dependency between them. At runtime, the topological visitor holds intermediate results from each input node with a temporary register buffer.

Implementation of Epilogue Operators. Figure 9 illustrates how the decoupled operators can be optimized and implemented separately. The original loop structure of all the operators is reconstructed into the same nested loops for optimal performance, with `parallel_for` tied to GPU thread hierarchies and `vector_for` for vectorized. For convenience, we name the code blocks at the start of the sequential loops as `begin_xxx`, and those at the end as `end_xxx`. The code block of the innermost sequential loop is called `visit`.

While all operators can be implemented by only filling the `visit` code block because of the perfectly nested loop constraint, better performance can be achieved by leveraging other blocks. For instance, in Figure 9(A), the element-wise load can be optimized by first copying a matrix fraction into registers at `begin_step` and then accessing them in `visit`.

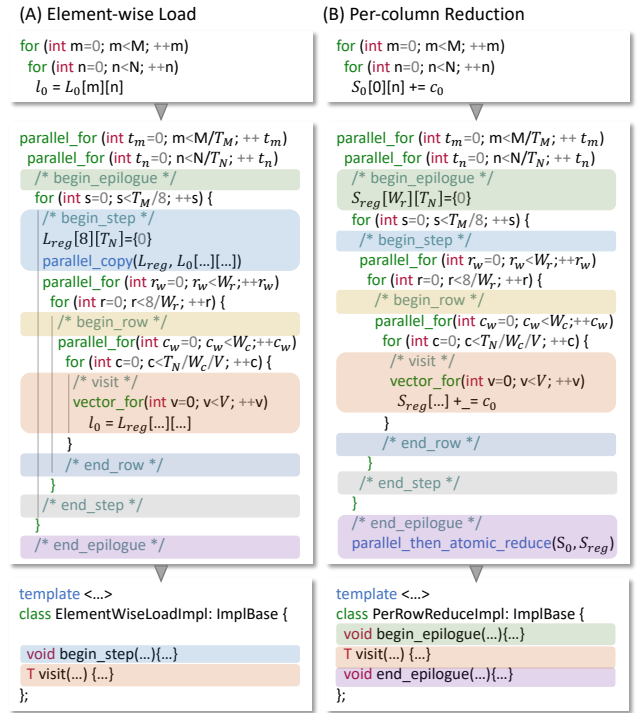


Figure 9. Implementation of Epilogue Operators

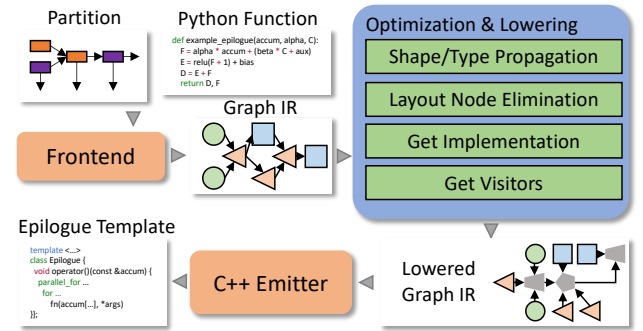


Figure 10. Overview of the EVT Operator Compiler

This ensures coalesced memory access and minimized address/predicate arithmetic. In Figure 9(B), the per-column reduction can be accelerated by initiating a temporary buffer at `begin_epilogue` to store partial results accumulated during `visit`, then perform the parallel reduction in `end_epilogue` before atomically reducing the result to global memory. Finally, as shown at the bottom of Figure 9, each operator can be implemented as a template class providing member functions defining the non-empty code blocks. These functions will be called by the visitors at runtime.

5.2 EVT Operator Compiler

With the Epilogue Visitor abstraction, we present a novel operator compiler Epilogue Visitor Tree (EVT). The "Tree"

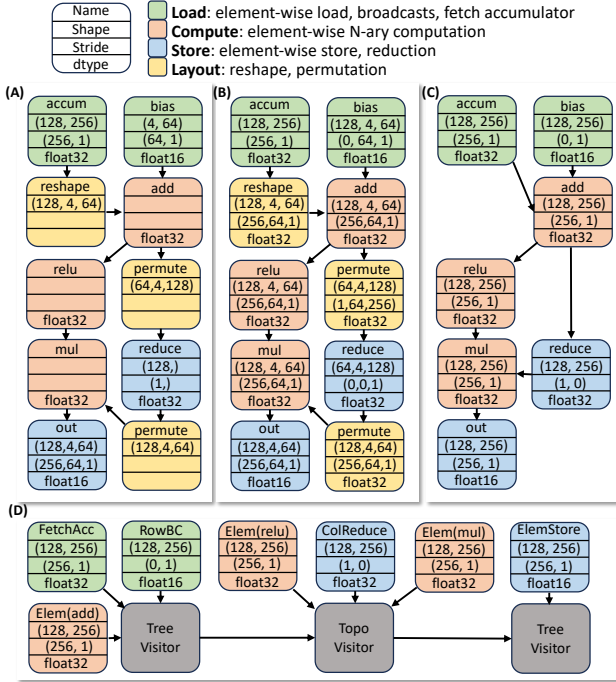


Figure 11. Example of EVT transformations.

comes from the fact that the transformed graph in Figure 8 (C) becomes a tree whose root is the last visitor. The overview of the compiler is shown in Figure 10.

The core of the compiler is a graph intermediate representation (IR) shown in Figure 11. It models the epilogue as a graph, where each node represents an operator. The nodes are classified into four types: load, compute, store, and layout. Each node also has four fields: name, shape, stride, and data type. The shape and stride are tuples following the CUTE library of CUTLASS¹. For instance, shape (m, n) and stride $(n, 1)$ represent a $m \times n$ matrix stored in the row-major layout. The index to the storage of the tensors in memory can be computed via inner product between shape and stride.

To support different input formats, the IR is constructed with minimum information and constraints by frontends. Then, the optimization and lowering schedules passes that fill in missing information and canonicalize the IR to meet the epilogue visitor abstraction. The C++ emitter then traverses the IR and emits the template class for the epilogue. This design allows adding new frontends for other compilation workflows easily while reusing the rest of the logic.

Frontend. EVT supports diverse user inputs from partitions generated by the previous section to Python functions. An example of the Python frontend is shown below, which defines the epilogue in Figure 11 (A).

```
def example_epilogue(accum, bias):
    add = reshape(accum, new_shape=(128, 4, 64)) + bias
    permute_1 = permute(add, indices=(2,1,0))
    reduce = sum(permute_1, dim=[0, 1])
    out = relu(add) * permute(permute_1, indices=(2,1,0))
    return reduce, out
```

```
tensors = {
    "accum": Tensor(torch.float32, shape=(128, 256)),
    "bias": Tensor(torch.float16, shape=(4, 64)),
    "reduce": Tensor(torch.float32, shape=(128,)),
    "out": Tensor(torch.float16, shape=(128, 4, 64))}
epilogue = python_ast.trace(example_epilogue, tensors)
```

Shape Type Propagation. This pass takes two stages. The first stage visits the nodes in topological order to fill in missing strides, shapes, and data types. The second stage, in reversed topological order, updates the shape and stride to remove implicit broadcast. For example, the shape and stride of *bias* are updated from $(4,64):(64,1)$ to $(128,4,64):(0,64,1)$, with the first dimension broadcasted.

Layout Node Elimination. This pass aims to canonicalize the IR by removing layout nodes. In the context of the epilogue visitor abstraction, all operators must align with the spacial loops of the producer, which can be translated to all the nodes having the same shape with the *accum* node with our IR. However, as shown in Figure 11 (B), layout nodes alter shapes and violate this requirement. We remove layout nodes by propagating them away from the *accum* through swapping with its neighbors. These layout nodes will eventually merge into a load or store node by updating their shape and stride. The result is shown in Figure 11 (D), all the operators now share the same shape.

Get Implementation and Visitor. This pass lowers the canonical IR to specific implementations. While our IR only has four types of nodes for simplicity, each type can have multiple underlying implementations for different strides. For instance, the *reduce* has stride $(1,0)$ that reduces the matrix into a column vector, while the *out* has stride $(256,1)$ that stores the matrix to memory in row-major. In this pass, we infer the implementation of each node based on its stride and inject the visitors for each non-leaf node based on its output degree. The result is shown in Figure 11 (D).

C++ Emitter. Our compiler provides a high-performance operator library including popular epilogue operators. The C++ Emitter emits the template class of the epilogue by simply traversing the IR in Figure 11 (D) and emitting each node in post-order. Notably, for operators not covered by our library, with our epilogue visitor abstraction, they can be implemented easily or generated by compiler techniques.

5.3 Additional Features

This section summarizes additional features EVT provides to improve flexibility and performance.

¹https://github.com/NVIDIA/cutlass/blob/main/media/docs/cute/01_layout.md

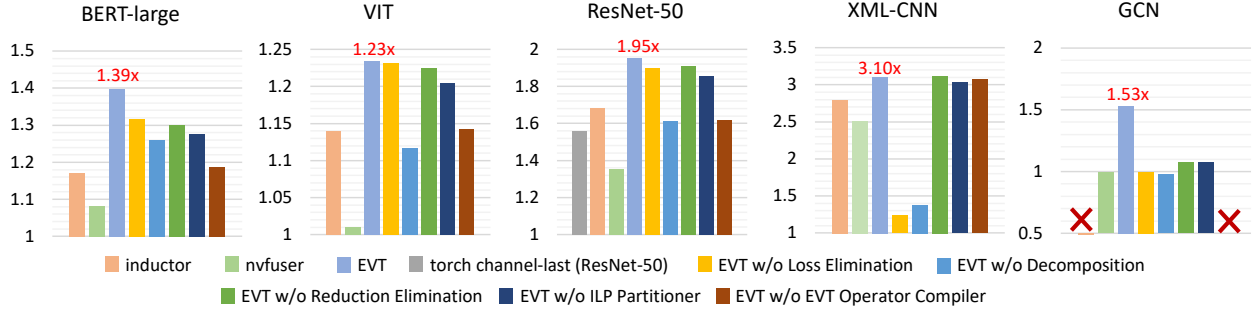


Figure 12. End-to-end Training Speedup over the PyTorch Implementation.

Dynamic Shapes. Our epilogue operators take the shape and stride as runtime parameters so that the same kernel compiled can support inputs with dynamic shapes.

Performance. EVT is compatible with the StreamK[20] for better workload balance. It also introduces the ping-pong buffer to the epilogue on Ampere GPU, enabling overlapping the computation and memory access in the epilogue.

Mainloop Fusion. While EVT focuses on epilogue fusion, it also offers the capability of fusing permutations of mainloop arguments for particular producers, such as matrix multiplication. In detail, CUTLASS indices the multiplicand and multiplier in matrix multiplication using the strides provided through its runtime arguments. We leverage this feature and recompute the stride of permuted inputs, which ensures the multiplicand and multiplier are accessed correctly without actually permuting their data in memory.

6 Evaluation

This section evaluates the performance of EVT on five real-world NN architectures and compares it with existing compilers. Section 6.2 evaluates the end-to-end training performance. Section 6.4 provides additional evaluations on representative layers of each architecture to explain the rationale behind EVT’s speedup. Besides, it also enables comparison with inference compilers such as TVM [4].

6.1 Experiment Setup

Benchmarks. We select BERT-Large [6], ViT [7], ResNet-50 [8], XML-CNN [17], and GCN [14] to cover diverse architectures and domains.

BERT-large [6] for language modeling and ViT [7] for computer vision is constructed using stacked self-attention layers, which have become an indispensable building block in modern neural networks. These two benchmarks cover multiple mainloops including GEMM, Batched GEMM, Softmax, and LayerNorm operations. Their epilogue consists of various element-wise operations, activation functions, reductions, reshapes, and permutations.

ResNet-50 [8] is one of the most popular backbone networks used in computer vision tasks. In addition to GEMM, ResNet-50 includes special heavy operations like Conv2dFprop and Conv2dDgrad. Additionally, the “Conv-BN-ReLU” pattern of ResNet-50 makes it challenging to be fused due to the complexity of BN operation.

XML-CNN [17] is a representative example of extreme classification tasks, where the label space contains millions of categories. Its performance is critical for applications such as language modeling and recommendation systems. Due to the enormous number of categories, XML-CNN presents new challenges in optimizing the loss function, which uses binary cross-entropy loss, its gradient kernel, as well as surrounding element-wise functions and reductions.

GCN [14] for node classification is selected as an example of graph neural networks (GNN) used to model graph-structured data, such as protein and social networks. GCN introduces a new mainloop operation, SpMM, for its aggregation stage. Additionally, the large batch size of whole graph training can pose challenges for U-turn optimization.

Platform. Single NVIDIA A100 GPU (40 GB) with CUDA 12.1 and NGC docker nvcr.io/nvidia/pytorch:23.07-py3.

Baselines. In the end-to-end benchmark, EVT is compared with Torch Inductor [22] and NVFuser. The former integrates various state-of-the-art compiler techniques from Triton [26] to CUTLASS[25]. We set the mode to *max-autotune* for optimal performance. The NVFuser is an operator compiler for GPU that just-in-time compiles fast and flexible GPU-specific code. In layer-wise benchmarks, we compare with Triton[26] and TVM [4]. For TVM, GEMM and convolutions are tuned by *autotvm* to leverage Tensor Cores, while others are generated through Ansor [31]. The CUDA Graph and AMP are applied to all methods for fair comparison.

6.2 End-to-End Benchmarks

The end-to-end training speedup for the five benchmarks is summarized in Figure 12 with results normalized to the naive PyTorch Implementation. Our EVT achieves 1.23~3.10× end-to-end training speedup across all benchmarks, outperforming existing compilers.

6.3 Ablation Study

To demonstrate the contribution of individual optimizations, Figure 12 also incorporates results from the ablation studies.

Ablation Study Setup. The ablation studies regarding "Loss Elimination", "Decomposition", and "Reduction Elimination" are performed by excluding the corresponding passes from the pass manager. In terms of the "ILP Partitioner", an alternative naive partitioner is used. Following existing studies, this partitioner first split the computation graph into forward and backward passes, and then find fusible subgraphs with simple heuristics inspired by TVM [4]. The ablation study for the "EVT Operator Compiler" incorporates graph-level optimizations into the torch inductor's backend, performing operator-level optimizations via the inductor. Notably, GCN is excluded as the inductor does not support sparse computation.

Contribution of Loss Elimination. The loss elimination pass has significant influence on models where the loss function and its backward pass contribute substantially to execution time. For example, in XML-CNN, it removes the Binary Cross-Entropy Loss. In GCN, it enables the optimization of the softmax with cross-entropy loss, as depicted in Figure 2. This demonstrates the effectiveness of loss elimination as an optimization strategy for the loss functions, absenting in prior studies.

Contribution of Decomposition. The decomposition creates additional fusion opportunities for fusion. Taking the ResNet-50 as an example, decomposing the batch normalization enables additional fusion opportunities, leading to the significant performance improvements shown in Figure 12.

Contribution of Reduction Elimination. In GCN, reduction elimination improves the performance by enabling the fusion between softmax and the decomposed nllLoss backward, as illustrated in Figure 2.

Contribution of Partitioner. The results in Figure 12 without the ILP-based partitioner have inferior performance compared to EVT across all the five models. Notably, besides performance consideration, the ILP-based formulation offers flexibility and extensibility to integrate additional objective functions, constraints, and heuristics.

Contribution of EVT Operator Compiler. Figure 12 illustrates that despite incorporating EVT's graph-level optimizations, the inductor still achieves inferior performance compared to EVT. This demonstrates that EVT's operator compiler generates fused kernels with superior performance.

6.4 Layer-Wise Benchmarks

This section evaluates EVT on representative layers from the five models. For each benchmark, we visualize the computation graph from Triton, TVM, and our EVT. Operators fused are grouped with boxes, with orange indicating heavy ops (e.g. GEMM, Softmax) and green indicating light ones

(element-wise / reductions). We also show the normalized latency over the PyTorch, which is divided into heavy and light ops. The speedups are also annotated for convenience.

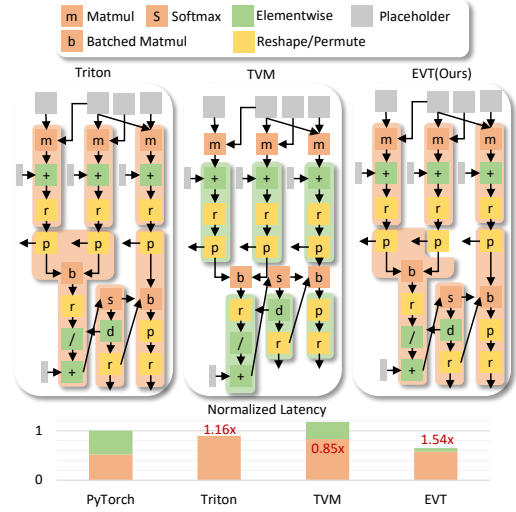


Figure 13. The Self-Attention Layer. (Bert, ViT)

Self-Attention Layer. Accelerating the self-attention layer requires the compiler to 1) efficiently handle different permutations and reshapes of the same tensor, and 2) generate high-performance fused kernels. EVT handles 1) through the *cute*-based IR design and passes in Section 5.2 and guarantees 2) through the Epilogue Visitor abstraction. In contrast, fused kernels generated by Triton are less efficient than ours, while TVM does not handle 1) efficiently.

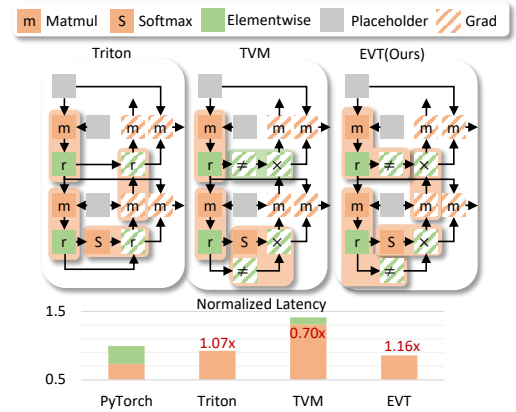


Figure 14. The Multilayer Perceptron (MLP). (Bert, ViT)

MLP. Optimizing the MLP requires the compiler to find the feasible and optimal partition of the computation graph. In Figure 14, EVT identifies the optimal cut between $\neq$ and $\times$ decomposed from the ReLU backward through the partitioner in Section 4, while Triton and TVM generate suboptimal partitions.

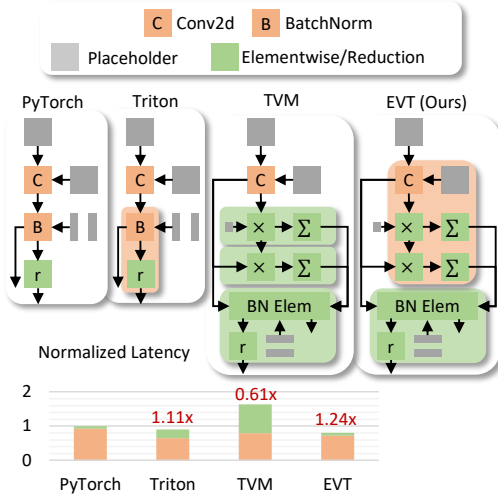


Figure 15. The Conv-BN-ReLU Layer. (ResNet)

Conv-BN-ReLU. The major challenge of BN under the channel-last layout is the strided access to the reduction dimension, so optimizing this layer requires compilers to 1) decompose the batch norm operator to create new fusion opportunities, and 2) generate high-performance kernels under the channel-last layout. EVT addresses this by decomposing BN to reduction and element-wise, and the reduction is fused to the preceding convolution efficiently. In contrast, Triton’s fused BN-ReLU is less performant due to the layout, and TVM fails to fuse the reductions.

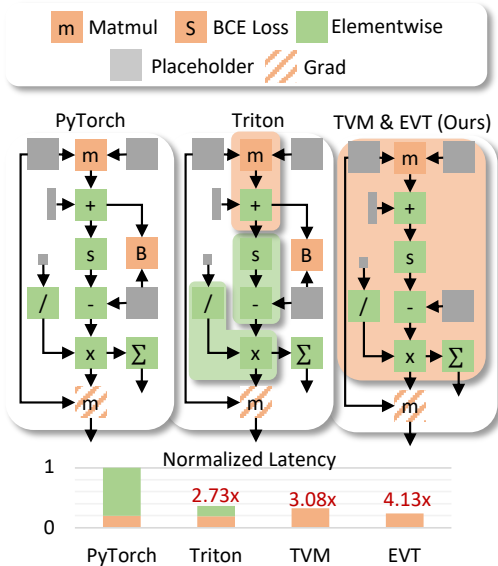


Figure 16. The Binary Cross-Entropy Loss. (XML-CNN)

Binary Cross-Entropy Loss. Accelerating BCE loss involves 1) identifying fusion opportunities on the fence of

the forward and backward pass and 2) generating high-performance fused GEMM with diverse epilogue operators, including broadcast, various element-wise operators, and reductions. EVT accomplishes 1) through its loss elimination pass and achieves 2) through the Epilogue Visitor abstraction. In contrast, Triton failed to cross the boundary between forward and backward graphs, while TVM creates less performant kernels.

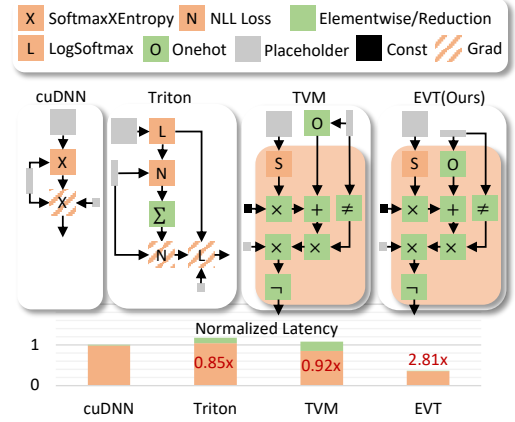


Figure 17. Softmax Cross-Entropy Loss. (GCN)

Softmax Cross-Entropy Loss. This graph shares similar optimization requirements with BCE loss. With the graph-level optimizations and operator compiler, EVT fuses the entire graph into a single kernel that offers 2.81× speedup over cuDNN. Oppositely, Triton failed to identify fusion opportunities between forward and backward graphs, while TVM creates less efficient kernels.

7 Conclusion & Discussion

EVT enables generating fused kernels with state-of-the-art performance while accommodating diverse fusion patterns. Along with the graph-level optimizations and partitioner that fully unleash its potential, EVT optimizes the deep learning training workload automatically and achieves state-of-the-art performance.

Impact on Multi-GPU/Node Training. While we majorly focus on a single GPU in this paper, EVT can be easily leveraged to accelerate the multi-GPU/node training. Particularly, a recent trend is fusing GEMMs with succeeding communication kernels that allow overlapping between computation and communication at the tile granularity [3, 23]. While this technique originally required manually implementing these fused kernels by experts, with EVT, the communication operator can be implemented as a special type of store node while reusing the rest of the EVT infrastructure.

References

- [1] Riyadh Baghdadi, Jessica Ray, Malek Ben Romdhane, Emanuele Del Sozzo, Abdurrahman Akkas, Yunming Zhang, Patricia Suriana, Shoaib Kamil, and Saman Amarasinghe. Tiramisu: A polyhedral compiler for expressing fast and portable code. In *2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, pages 193–205. IEEE, 2019.
- [2] Preston Briggs and Keith D Cooper. Effective partial redundancy elimination. *ACM SIGPLAN Notices*, 29(6):159–170, 1994.
- [3] Tapabrata Chakraborti, Brendan McCane, Steven Mills, and Umapada Pal. Coconet: A collaborative convolutional network. *arXiv preprint arXiv:1901.09886*, 2019.
- [4] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Meghan Cowan, Haichen Shen, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. Tvm: an automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX conference on Operating Systems Design and Implementation*, pages 579–594, 2018.
- [5] Zhaodong Chen, Zheng Qu, Liu Liu, Yufei Ding, and Yuan Xie. Efficient tensor core-based gpu kernels for structured sparsity under reduced precision. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–14, 2021.
- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 4171–4186, 2019.
- [7] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*.
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [9] Julien Herrmann, Jonathan Kho, Bora Uçar, Kamer Kaya, and Ümit V Çatalyürek. Acyclic partitioning of large directed acyclic graphs. In *2017 17th IEEE/ACM international symposium on cluster, cloud and grid computing (CCGRID)*, pages 371–380. IEEE, 2017.
- [10] Guyue Huang, Guohao Dai, Yu Wang, and Huazhong Yang. Gspmm: General-purpose sparse matrix-matrix multiplication on gpus for graph neural networks. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–12. IEEE, 2020.
- [11] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015.
- [12] Wonkyung Jung, Daejin Jung, Byeongho Kim, Sunjung Lee, Wonjong Rhee, and Jung Ho Ahn. Restructuring batch normalization to accelerate cnn training. *Proceedings of Machine Learning and Systems*, 1:14–26, 2019.
- [13] Brian W Kernighan and Shen Lin. An efficient heuristic procedure for partitioning graphs. *The Bell system technical journal*, 49(2):291–307, 1970.
- [14] Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*.
- [15] Chris Lattner, Mehdi Amini, Uday Bondhugula, Albert Cohen, Andy Davis, Jacques Pienaar, River Riddle, Tatiana Shpeisman, Nicolas Vasilache, and Oleksandr Zinenko. Mlir: A compiler infrastructure for the end of moore’s law. *arXiv preprint arXiv:2002.11054*, 2020.
- [16] Mingzhen Li, Yi Liu, Xiaoyan Liu, Qingxiao Sun, Xin You, Hailong Yang, Zhongzhi Luan, Lin Gan, Guangwen Yang, and Depei Qian. The deep learning compiler: A comprehensive survey. *IEEE Transactions on Parallel and Distributed Systems*, 32(3):708–727, 2020.
- [17] Jingzhou Liu, Wei-Cheng Chang, Yuexin Wu, and Yiming Yang. Deep learning for extreme multi-label text classification. In *Proceedings of the 40th international ACM SIGIR conference on research and development in information retrieval*, pages 115–124, 2017.
- [18] Clair E Miller, Albert W Tucker, and Richard A Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM (JACM)*, 7(4):326–329, 1960.
- [19] Jenny Nossack and Erwin Pesch. A branch-and-bound algorithm for the acyclic partitioning problem. *Computers & operations research*, 41:174–184, 2014.
- [20] Muhammad Osama, Duane Merrill, Cris Cecka, Michael Garland, and John D Owens. Stream-k: Work-centric parallel decomposition for dense matrix-matrix multiplication on the gpu. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, pages 429–431, 2023.
- [21] M Yusuf Özkaya and Ümit V Çatalyürek. A simple and elegant mathematical formulation for the acyclic dag partitioning problem. *arXiv preprint arXiv:2207.13638*, 2022.
- [22] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché Buc, E. Fox, and R. Garnett, editors, *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. URL: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
- [23] Kishore Punniyamurthy, Bradford M Beckmann, and Khaled Hamidouche. Gpu-initiated fine-grained overlap of collective communication with computation. *arXiv preprint arXiv:2305.06942*, 2023.
- [24] Nino Shervashidze, Pascal Schweitzer, Erik Jan Van Leeuwen, Kurt Mehlhorn, and Karsten M Borgwardt. Weisfeiler-lehman graph kernels. *Journal of Machine Learning Research*, 12(9), 2011.
- [25] Vijay Thakkar, Pradeep Ramani, Cris Cecka, Aniket Shivam, Honghao Lu, Ethan Yan, Jack Kosaian, Mark Hoemmen, Haicheng Wu, Andrew Kerr, Matt Nicely, Duane Merrill, Dustyn Blasig, Fengqi Qiao, Piotr Majcher, Paul Springer, Markus Hohnerbach, Jin Wang, and Manish Gupta. CUTLASS, 1 2023. URL: <https://github.com/NVIDIA/cutlass>.
- [26] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, pages 10–19, 2019.
- [27] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. Tensor comprehensions: Framework-agnostic high-performance machine learning abstractions. *arXiv preprint arXiv:1802.04730*, 2018.
- [28] Guibin Wang, YiSong Lin, and Wei Yi. Kernel fusion: An effective method for better power efficiency on multithreaded gpu. In *2010 IEEE/ACM Int’l Conference on Green Computing and Communications & Int’l Conference on Cyber, Physical and Social Computing*, pages 344–350. IEEE, 2010.
- [29] Jiarong Xing, Leyuan Wang, Shang Zhang, Jack Chen, Ang Chen, and Yibo Zhu. Bolt: Bridging the gap between auto-tuners and hardware-native performance. *Proceedings of Machine Learning and Systems*, 4:204–216, 2022.
- [30] Bing Xu, Ying Zhang, Hao Lu, Yang Chen, Terry Chen, Mike Iovine, Mu-Chu Lee, and Zhijiang Li. AITemplate, 10 2022. URL: <https://github.com>.

com/facebookincubator/AITemplate.

- [31] Lianmin Zheng, Chengfan Jia, Minmin Sun, Zhao Wu, Cody Hao Yu, Ameer Haj-Ali, Yida Wang, Jun Yang, Danyang Zhuo, Koushik Sen, et al. Ansor: Generating high-performance tensor programs for deep learning. In *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation*, pages 863–879, 2020.

A Artifact Appendix

A.1 Abstract

This artifact contains the source code of EVT, including the Dockerfile to setup the environment and bash scripts to reproduce the speedups shown in Figure 12 - 17.

A.2 Artifact check-list (meta-information)

- **Program:** The *EVT_AE/python* contains the source code of the EVT compiler. *EVT_AE/src* includes additional CUDA source code. The benchmarks used for evaluation can be found in *EVT_AE/benchmark*.
- **Data set:** ogbn-mag.
- **Run-time environment:** All the dependencies are provided through Dockerfile. The results are evaluated under driver version 530.30.02, CUDA 12.1.
- **Hardware:** A single A100-SXM4-40GB is required.
- **Output:** Two figures similar to Figure 12 and Figure 13-17 will be generated under the *EVT_AE/* directory.
- **Experiments:** Bash scripts *figure12.sh* and *figure13_17.sh* are included. *EVT_AE/README.md* provides more detailed instructions.
- **How much disk space required (approximately)?:** less than 50 GB.
- **How much time is needed to prepare workflow (approximately)?:** Less than 20 minutes to build the docker image.
- **How much time is needed to complete experiments (approximately)?:** Around 2 hours.
- **Publicly available?:** Yes.

A.3 Description

A.3.1 How to access. The source code is publicly available at https://github.com/apuaaChen/EVT_AE/tree/main and <https://zenodo.org/doi/10.5281/zenodo.10790585>.

A.3.2 Hardware dependencies. To reproduce the results, a single A100 GPU with at least 40GB HBM is required. The CPU should not influence the results, but may have impact on the compilation time.

A.3.3 Software dependencies. The Dockerfile provided is built from docker image *nvcr.io/nvidia/pytorch:23.07-py3*. Other dependencies are specified in the Dockerfile. Notably, small changes are automatically applied to the source code of PyTorch to workaround certain issues when tracing GCNs with *torch.compile*.

A.3.4 Data sets. The GCN is evaluated on the ogbn-mag dataset. Others use synthesized inputs.

A.4 Installation

The detailed installation steps are in *EVT_AE/README.md*.

A.5 Experiment workflow

Follow the steps in *EVT_AE/README.md*, launch the two scripts: 'bash figure12.sh' and 'bash figure13_17.sh'.

A.6 Evaluation and expected results

Two figures will be generated under the *EVT_AE* directory after the evaluation is done. These two figures summarize the relative speedup achieved by EVT and other baseline implementations in Figure 12-17 of the original paper. The exact speedup should be close to what reported in the paper, but small variance may exist. The *EVT_AE/README.md* also includes the reference results obtained on our own server.